



Министерство науки и высшего образования Российской Федерации
Федеральное государственное
бюджетное образовательное учреждение высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ

«Фундаментальные Науки»

КАФЕДРА

ФН-12 «Математическое моделирование»

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ НА ТЕМУ:

Редакционные расстояния

Студент:

Подобедов В. В.

дата, подпись

Ф.И.О.

Преподаватель:

Волкова Л. Л.

дата, подпись

Ф.И.О.

Москва, 2023

Содержание

Введение	3
1 Аналитическая часть	4
2 Конструкторская часть	5
3 Технологическая часть	8
4 Исследовательская часть	17
Заключение	19
Список литературы	20

Введение

Цель лабораторной работы: провести сравнительный анализ алгоритмов поиска редакционных расстояний.

Для достижения поставленной цели требуется решить следующие **задачи**.

1. Описать расстояния Левенштейна и Дameraу-Левенштейна.
2. Разработать алгоритмы вычисления редакционных расстояний, согласно варианту.
3. Реализовать разработанные алгоритмы.
4. Выполнить тестирование реализации разработанных алгоритмов.
5. Разработать программное обеспечение с двумя режимами работы — режимом расчёта расстояний для введённых пользователем двух строк произвольной длины и режимом массивированного замера процессорного времени для автоматически сгенерированных строк заданной длины.
6. Провести сравнение временной эффективности реализации разработанных алгоритмов, проведя замеры процессорного времени их выполнения в зависимости от линейного размера строк одинаковой длины.
7. Выполнить оценку ёмкостной эффективности разработанных алгоритмов в зависимости от линейных размеров строк (для рекурсивных алгоритмов дать оценку пиковой затрачиваемой памяти) Провести сравнение временной эффективности реализации разработанных алгоритмов, проведя замеры памяти, затрачиваемой реализацией разработанных алгоритмов в зависимости от линейных размеров строк (для рекурсивных алгоритмов замерить пиковую затрачиваемую память).

Согласно варианту, требуется разработать следующие три алгоритма.

1. Алгоритм вычисления расстояния Левенштейна нерекурсивным методом, используя две строки.
2. Алгоритм вычисления расстояния Дameraу-Левенштейна нерекурсивным методом, пользуясь матрицей и в конце программы вывести её содержимое.
3. Алгоритм вычисления расстояния Дameraу-Левенштейна рекурсивным методом без кэша.

1 Аналитическая часть

Расстояние между строками а и b можно найти при помощи некоторых редакционных расстояний. Рассмотрим их.

Расстояние Левенштейна — это метрика сходства между двумя строковыми последовательностями [1]. Оно может быть рассчитано по формуле 1:

$$d_{a,b}(i, j) = \begin{cases} 0 & i = 0, j = 0 \\ i & j = 0, i > 0 \\ j & i = 0, j > 0 \\ \min \begin{cases} d_{a,b}(i, j - 1) + 1 \\ d_{a,b}(i - 1, j) + 1 \\ d_{a,b}(i - 1, j - 1) + m(a[i], b[j]) \end{cases} & j > 0, i > 0 \end{cases} \quad (1)$$

Нетрудно понять, что первые три строчки формулы заполняют первый столбец и первую строку матрицы. Для всех остальных ячеек матрицы используется четвертая формула. Стоит уделить внимание оператору “ $m(a[i], b[j])$ ”. Он сравнивает символы в слове, и, если они одинаковые, он дает 0, иначе — 1.

Расстояние Дамерау-Левенштейна вычисляется по формуле 2:

$$d_{a,b}(i, j) = \begin{cases} \max(i, j) & \text{если } i \cdot j = 0; \\ \min \begin{cases} d_{a,b}(i - 1, j) + 1 \\ d_{a,b}(i, j - 1) + 1 \\ d_{a,b}(i - 1, j - 1) + 1_{(a_i=b_j)} \\ d_{a,b}(i - 2, j - 2) + 1 \end{cases} & \text{если } i, j > 1, a_i = b_{j-1}, a_{i-1} = b_j; \\ \min \begin{cases} d_{a,b}(i - 1, j) + 1 \\ d_{a,b}(i, j - 1) + 1 \\ d_{a,b}(i - 1, j - 1) + 1_{(a_i=b_j)} \end{cases} & \text{иначе} \end{cases} \quad (2)$$

Значительным отличием в нахождении расстояния Дамерау-Левенштейна от расстояния Левенштейна является транспозиция соседних символов.

Говоря об эффективности методов вычисления расстояния слов, хочется отметить рекурсивный способ Дамерау-Левенштейна с кэшем. Поскольку после каждого подсчета порождается еще три вычисления, рекурсия с кэшем может помочь сохранить процессорное время. Поэтому вместо $3^{k \cdot l}$ раз, где k и l длина слов соответственно, нужно вычислить $k \cdot l$ раз. Что касается других способов решения задачи, можно использовать нерекурсивный метод с заполнением матрицы размера $(k + 1) \cdot (l + 1)$. Однако можно заметить, что для вычисления расстояния Левенштейна используются только числа предыдущей и вычисляемой строки. Поэтому в каждый момент вычисления можно хранить две строки матрицы.

2 Конструкторская часть

Схема алгоритма поиска расстояния Левенштейна нерекурсивным способом с хранением двух строк матрицы представлена на рис. 1.



Рисунок 1 – Схема алгоритма поиска расстояния Левенштейна нерекурсивным способом

Схема алгоритма поиска расстояния Дамерау-Левенштейна нерекурсивным способом с хранением матрицы представлена на рис. 2.



Рисунок 2 – Схема алгоритма поиска расстояния Дамерау-Левенштейна нерекурсивным способом с хранением матрицы

Схема алгоритма поиска расстояния Дамерау-Левенштейна рекурсивным способом с хранением 3 строк матрицы представлена на рис. 3.

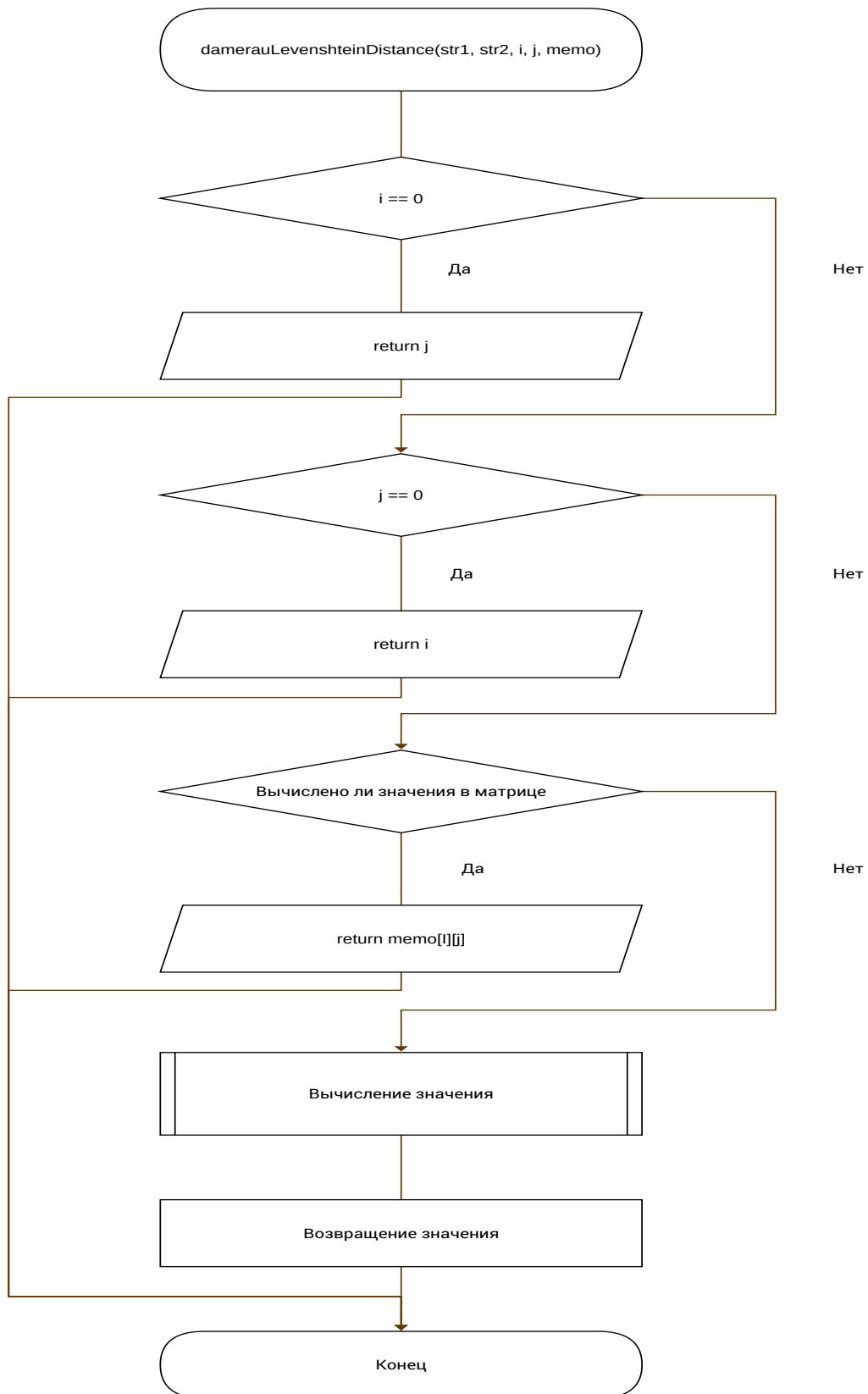


Рисунок 3 – Схема алгоритма поиска расстояния Дамерау-Левенштейна рекурсивным способом с хранением 3 строк матрицы

3 Технологическая часть

Для реализации алгоритмов был выбран язык программирования C++. Кроме стандартной библиотеки `<iostream>`, которая предоставляет возможность пользоваться средствами ввода-вывода для стандартной консоли. Библиотека `<algorithm>` подключена для использования функции `min()`. Также для генерации различных слов от времени была заимствована библиотека `<ctime>`.

Функции, разработанные в ходе написания лабораторной работы.

1. `levenstein` — функция, которая рекурсивно вычисляет расстояние двух слов, используя две строки по алгоритму Левенштейна.
2. `DamerauLevenshteinDistance` — функция, которая рекурсивно вычисляет расстояние двух строк, используя для этого матрицу по алгоритму Дамерау-Левенштейна.
3. `damerauLevenshteinDistance` — функция, которая рекурсивно вычисляет расстояние двух слов по алгоритму Дамерау-Левенштейна с проверкой на кэш в матрице.

В программе реализовано переключение между двумя режимами: пользовательским и автоматическим. Первый способ дает возможность человеку самому ввести необходимые слова и узнать их расстояние. Второй способ дает возможность отследить ресурсоёмкость каждого из трех алгоритмов.

Далее представлены листинги 1, 2, 3 реализаций трех алгоритмов.

Листинг 1 – Расстояние Левенштейна рекурсивный метод, используя две строки матрицы

```
#include <iostream>
#include <string>
#include <vector>
#include <algorithm>
#include <ctime>

using namespace std;

string RandomWord(int length) {
    static const char alphabet[] = "абвгдеёжзиклмнопрстуфхцчщъыьэя";
    string result;

    for (int i = 0; i < length; ++i) {
        int randomIndex = rand() % (sizeof(alphabet) - 1);
        result += alphabet[randomIndex];
    }

    return result;
}
```



```

int levenstein(string str_1, string str_2) {
    int n = str_1.length();
    int m = str_2.length();

    vector<int> current_row(n + 1);
    for (int i = 1; i <= n; i++) {
        current_row[i] = i;
    }

    for (int i = 1; i <= m; i++) {
        vector<int> previous_row = current_row;
        current_row[0] = i;
        for (int j = 1; j <= n; j++) {
            current_row[j] = min(previous_row[j] + 1, min(
                current_row[j - 1] + 1,
                (str_1[j - 1] != str_2[i - 1]) ?
                    previous_row[j - 1] + 1 : previous_row
                    [j - 1]));
        }
    }
    return current_row[n];
}

int main() {
    setlocale(0, "");
    string str1, str2;
    srand(time(0));
    int a, k, num;
    double medtime = 0;
    cout << "Ввести данные вручную (1) или сгенерировать (2) - ";
    cin >> a;
    if (a == 1) {
        cout << "Введите первое слово : ";
        cin >> str1;
        cout << "Введите второе слово : ";
        cin >> str2;
        clock_t start_time = clock();
        cout << "Расстояние левенштейна: " << levenstein(str1, str2)
            << endl;
        clock_t end_time = clock();
        double cpu_time = (double(end_time - start_time)) /

```

```

        CLOCKS_PER_SEC;
        cout << "Процессорное время: " << cpu_time << endl;
        cout << endl;
    }
    else {
        cout << "Введите длинуслова : ";
        cin >> k;
        cout << "Количество слов: ";
        cin >> num;
        for (int i = 1; i <= num; i++) {
            cout << i << endl;
            str1 = RandomWord(k);
            str2 = RandomWord(k);
            cout << str1 << endl << str2 << endl;
            clock_t start_time = clock();
            cout << "Расстояние левенштейна: " << levenstein(
                str1, str2) << endl;
            clock_t end_time = clock();
            double cpu_time = (double(end_time - start_time))
                / CLOCKS_PER_SEC;
            medtime += cpu_time;
            cout << "Процессорное время: " << cpu_time << endl;
            cout << endl;
        }
        cout << "Среднее значение работы процессора : " << medtime /
            num;
    }
    return 0;
}

```

Листинг 2 – Расстояние Дameraу-Левенштейна нерекурсивный метод, используя матрицу

```

#include <iostream>
#include <string>
#include <algorithm>
#include <vector>
#include <ctime>

using namespace std;

string RandomWord(int length) {
    static const char alphabet[] = "абвгдеёжзиклмнопрстуфхцщъыьэюя";
    string result;

```

```

    for (int i = 0; i < length; ++i) {
        int randomIndex = rand() % (sizeof(alphabet) - 1);
        result += alphabet[randomIndex];
    }

    return result;
}

int DamerauLevenshteinDistance(const string& source, const string& target
    ) {

    int n = source.length();
    int m = target.length();
    vector<vector<int>> distanceMatrix(n + 1, vector<int>(m + 1));

    for (int i = 0; i <= n; ++i) {
        distanceMatrix[i][0] = i;
    }
    for (int j = 0; j <= m; ++j) {
        distanceMatrix[0][j] = j;
    }

    for (int i = 1; i <= n; ++i) {
        for (int j = 1; j <= m; ++j) {

            int substitutionCost = (source[i - 1] != target[j
                - 1]);

            int deletionCost = distanceMatrix[i - 1][j] + 1;
            int insertionCost = distanceMatrix[i][j - 1] + 1;
            int substitutionOrEqualityCost = distanceMatrix[i
                - 1][j - 1] + substitutionCost;

            // транспозиция , еслиэтовозможно
            if (i > 1 && j > 1 && source[i - 1] == target[j -
                2] && source[i - 2] == target[j - 1]) {
                distanceMatrix[i][j] = min({ deletionCost
                    , insertionCost,
                    substitutionOrEqualityCost,
                    distanceMatrix[i - 2][j - 2] +
                    substitutionCost });
            }
        }
    }
}

```

```

        }
        else {
            distanceMatrix[i][j] = min({ deletionCost
            , insertionCost ,
            substitutionOrEqualityCost });
        }
    }
}

for (int i = 0; i <= source.length(); ++i) {
    for (int j = 0; j <= target.length(); ++j) {
        cout << distanceMatrix[i][j] << " ";
    }
    cout << endl;
}

return distanceMatrix[n][m];
}

int main() {
    setlocale(0, "");
    string str1, str2;
    srand(time(0));
    int a, k, num;
    double medtime = 0;
    cout << "Ввести данные вручную (1) или сгенерировать (2) - ";
    cin >> a;
    if (a == 1) {
        cout << "Введите первое слово : ";
        cin >> str1;
        cout << "Введите второе слово : ";
        cin >> str2;
        clock_t start_time = clock();
        cout << "Расстояние ДамерауЛевенштейна -: " <<
            DamerauLevenshteinDistance(str1, str2) << endl;
        clock_t end_time = clock();
        double cpu_time = (double(end_time - start_time)) /
            CLOCKS_PER_SEC;
        cout << "Процессорное время: " << cpu_time << endl;
        cout << endl;
    }
    else {

```

```

        cout << "Введите длину слова : ";
        cin >> k;
        cout << "Количество слов: ";
        cin >> num;
        for (int i = 1; i <= num; i++) {
            cout << i << endl;
            str1 = RandomWord(k);
            str2 = RandomWord(k);
            cout << str1 << endl << str2 << endl;
            clock_t start_time = clock();
            cout << "Расстояние Дамерау-Левенштейна-: " <<
                DamerauLevenshteinDistance(str1, str2) << endl
                ;
            clock_t end_time = clock();
            double cpu_time = (double(end_time - start_time))
                / CLOCKS_PER_SEC;
            medtime += cpu_time;
            cout << "Процессорное время: " << cpu_time << endl;
            cout << endl;
        }
        cout << "Среднее значение работы процессора : " << medtime /
            num;
    }
    return 0;
}

```

Листинг 3 – Расстояние Дамерау-Левенштейна рекурсивный метод, используя три строки матрицы

```

#include <iostream>
#include <string>
#include <vector>
#include <algorithm>
#include <ctime>

using namespace std;

string RandomWord(int length) {
    static const char alphabet[] = "абвгдеёжзиклмнопрстуфхцчшщъыьэя";
    string result;

    for (int i = 0; i < length; ++i) {
        int randomIndex = rand() % (sizeof(alphabet) - 1);
    }
}

```

```

        result += alphabet[randomIndex];
    }

    return result;
}

int damerauLevenshteinDistance(const string& str1, const string& str2,
    int i, int j, vector<vector<int>>& memo) {
    if (i == 0)
        return j;

    if (j == 0)
        return i;

    if (memo[i][j] != -1)
        return memo[i][j];

    int cost = (str1[i - 1] == str2[j - 1]) ? 0 : 1;

    int deletee = damerauLevenshteinDistance(str1, str2, i - 1, j,
        memo) + 1;
    int insert = damerauLevenshteinDistance(str1, str2, i, j - 1,
        memo) + 1;
    int substitution = damerauLevenshteinDistance(str1, str2, i - 1,
        j - 1, memo) + cost;

    int distance = min({ deletee, insert, substitution });

    if (i > 1 && j > 1 && str1[i - 1] == str2[j - 2] && str1[i - 2]
        == str2[j - 1]) {
        int transposition = damerauLevenshteinDistance(str1, str2
            , i - 2, j - 2, memo) + cost;
        distance = min(distance, transposition);
    }

    memo[i][j] = distance;
    return distance;
}

int damerauLevenshteinDistance(const string& str1, const string& str2) {
    int m = str1.length();
    int n = str2.length();

```

```

vector<vector<int>> memo(m + 1, vector<int>(n + 1, -1));

return damerauLevenshteinDistance(str1, str2, m, n, memo);
}

int main() {
    setlocale(0, "");
    string str1, str2;
    srand(time(0));
    int a, k, num;
    double medtime = 0;
    cout << "Ввести данные вручную (1) или сгенерировать (2) - ";
    cin >> a;
    if (a == 1) {
        cout << "Введите первое слово : ";
        cin >> str1;
        cout << "Введите второе слово : ";
        cin >> str2;
        clock_t start_time = clock();
        cout << "Расстояние ДамерауЛевенштейна-: " <<
            damerauLevenshteinDistance(str1, str2) << endl;
        clock_t end_time = clock();
        double cpu_time = (double(end_time - start_time)) /
            CLOCKS_PER_SEC;
        cout << "Процессорное время: " << cpu_time << endl;
        cout << endl;
    }
    else {
        cout << "Введите длину слова : ";
        cin >> k;
        cout << "Количество слов: ";
        cin >> num;
        for (int i = 1; i <= num; i++) {
            cout << i << endl;
            str1 = RandomWord(k);
            str2 = RandomWord(k);
            cout << str1 << endl << str2 << endl;
            clock_t start_time = clock();
            cout << "Расстояние ДамерауЛевенштейна-: " <<
                damerauLevenshteinDistance(str1, str2) << endl
                ;
        }
    }
}

```

```

        clock_t end_time = clock();
        double cpu_time = (double(end_time - start_time))
            / CLOCKS_PER_SEC;
        medtime += cpu_time;
        cout << "Процессорное время: " << cpu_time << endl;
        cout << endl;
    }
    cout << "Среднее значение работы процессора : " << medtime /
        num;
}

return 0;

}

```


4 Исследовательская часть

В таблице 1 приведено время в секундах, потраченное на вычисление расстояния определённым алгоритмом:

Алгоритм	5 символов	10 символов	15 символов	20 символов	50 символов
Левенштейн без рекурсии 2 строки	0.002	0.0026	0.0028	0.004	0.0097
Дамерау-Левенштейн без рекурсии матрица	0.012	0.029	0.063	0.116	0.6396
Дамерау-Левенштейн рекурсия с кэшем	0.002	0.003	0.004	0.0058	0.0252

Таблица 1

Результаты вычислений также представлены на графике (рисунок 4):

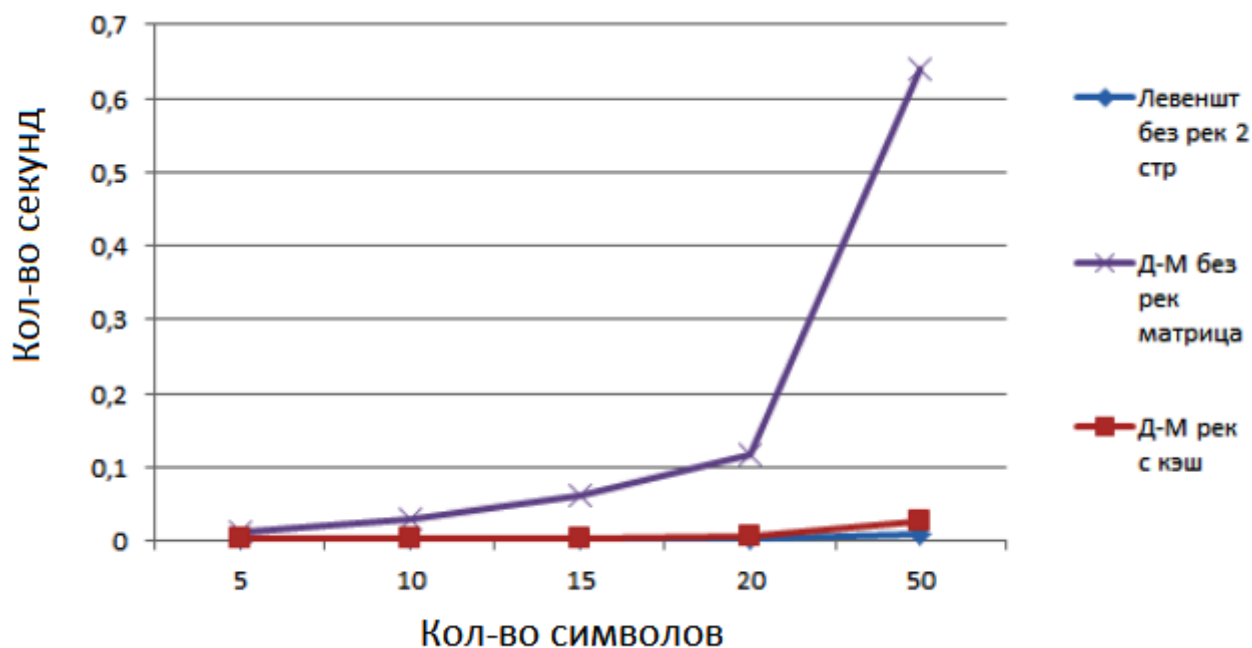


Рисунок 4 – График зависимости затраченного времени

Нетрудно заметить, что затраченное время возрастает с увеличением количества вычислений. Поскольку при заполнении матрицы требуется сделать $3^{n \cdot n}$ вычислений, где n - количество символов в слове, то необходимо расходовать большое количество времени. Что касается алгоритма Дамерау-Левенштейна с рекурсией с кэшем, то затраченного времени и

расходованной памяти, безусловно, становится гораздо меньше. Однако при больших объемах информации лидирует алгоритм Левенштейна без рекурсии с помощью двух строк. Данный алгоритм универсален, так как хранит в разы меньше информации по сравнению с другими алгоритмами и тратит на вычисления меньше времени. Он будет являться самым “выигрышным”.

Заключение

В результате выполнения лабораторной работы:

1. Описано расстояние Левенштейна и Дамерау-Левенштейна.
2. Разработаны алгоритмы вычисления редакционных расстояний, согласно варианту.
3. Реализованы разработанные алгоритмы.
4. Выполнено тестирование реализации разработанных алгоритмов.
5. Разработано программное обеспечение с двумя режимами работы — режимом расчёта расстояний для введённых пользователем двух строк произвольной длины и режимом массивованного замера процессорного времени для автоматически сгенерированных строк заданной длины.
6. Проведено сравнение временной эффективности реализации разработанных алгоритмов.
7. Выполнена оценка емкостной эффективности разработанных алгоритмов в зависимости от линейных размеров строк.

Согласно варианту, разработано следующие три алгоритма.

1. Алгоритм вычисления расстояния Левенштейна нерекурсивным методом, используя две строки.
2. Алгоритм вычисления расстояния Дамерау-Левенштейна нерекурсивным методом, используя матрицы и в конце программы вывести её содержимое.
3. Алгоритм вычисления расстояния Дамерау-Левенштейна рекурсивным методом без кэша.

После обработки полученных данных наиболее эффективным алгоритмом оказался Левенштейна без рекурсии в две строки. Он меньше всего затрачивает память компьютера, что делает его максимально эффективным по сравнению с другими алгоритмами.

Список литературы

- [1] Левенштейн В. И. Двоичные коды с исправлением выпадений, вставок и замещений символов / Доклады АН СССР, т. 163. — М.: Издательство «Наука», 1965.